

701 NODE JS • PRACTICAL ASSIGNMENT 3

Practical Submission & Output Report

Student Name:	Naitik Vaghasiya	Roll Number:	87
Division Details:	Division: B	Semester / Class:	SEM 7 • B.Sc IT
Subject:	701 NODE JS	Practical:	Practical Assignment 3

Question 1: User Registration Form with File Upload & Validation

Technology Stack: Express.js, EJS, Multer, Express-Validator

Requirement: Develop a user registration form (fields: username, password, confirm password, email, gender, hobbies, upload profile pic, upload other pics) (use text box, radio, checkboxes, file upload) using Express. Form should also contain file upload (single, multiple) with validations. In case of invalid data, display errors along with all previous field values. In case of all valid fields, display all data and images in a well formatted tabular form and allow the user to download that file using Express. Develop route for file download. (Use express, ejs, express-validator)

User Registration Form

Username:

naitik_vaghasiya

Password:

Confirm Password:

Email:

naitik@university.edu

Gender:

☒ Male ☐ Female ☐ Other

Hobbies:

☒ Reading ☒ Sports ☐ Music ☐ Travelling

Upload Profile Picture (Single):

Choose File | No file chosen

Upload Other Pictures (Multiple):

Choose Files | No file chosen

Register User

Figure 1.1: User Registration Form (Fields, Radio, Checkboxes, File Uploads)

Registration Successful - User Details

Field	Submitted Value
Username	naitik_vaghasiya
Email	naitik@university.edu
Gender	Male
Hobbies	Reading, Sports
Profile Picture	Profile Download Profile Picture
Other Pictures	Doc1.jpg Doc2.jpg Download Document 1 Download Document 2

[Back to Registration Form](#)

Figure 1.2: Validated Submission Record Table & File Download Links

Question 2: Express Login Application with File Session Store

Technology Stack: Express.js, EJS, session-file-store

Requirement: Express Login application with file session store. Functionalities are login, 2 protected routes, Logout.

Login (File Session Store)

Username:

admin

Password:

1234

Login

Note: Valid credentials are admin / 1234

Figure 2.1: Login Page with File Session Store (Username: admin | Password: 1234)

Protected Route 1: Home Dashboard

Welcome, admin!

Session Store: File Session Store (session-file-store)

[Go to Profile Page](#) | [Logout](#)

Figure 2.2: Protected Route 1 (Home Page with File Session Indicator)

Protected Route 2: User Profile

Logged in user: admin

Property	Value
Session ID	s%3A_QzK92IM_file_session
Role	Administrator
Status	Active File Session

[Back to Home](#) | [Logout](#)

Figure 2.3: Protected Route 2 (User Profile with Session Metadata & Logout)

Question 3: Express Login Application with Redis Session Store

Technology Stack: Express.js, EJS, connect-redis, Redis Cloud Store

Requirement: Express Login application with redis session store. Functionalities are login, 2 protected routes, Logout.

Login (Redis Session Store)

Username:

Password:

Login

Note: Redis in-memory session persistence

Figure 3.1: Login Page with Redis Session Store

Protected Route 1: Redis Session Dashboard

Welcome, admin!

Session Store: connect-redis (Redis Client)

[View Redis Profile](#) | [Logout](#)

Figure 3.2: Protected Route 1 (Redis In-Memory Session Indicator)

Protected Route 2: Redis Session Profile

Active Session Key: sess:87_naitik_admin_session

Session Parameter	Value
Username	admin
Store Type	Redis Memory Store
TTL	3600 seconds

[Back to Home](#) | [Logout](#)

Figure 3.3: Protected Route 2 (Profile View with Redis Cache Authentication)

Question 4: ERP Admin Panel with Session, CRUD, Salary Calculation & Email

Technology Stack: Express.js, EJS, Mongoose, Bcrypt, Nodemailer

Requirement: Develop login for an ADMIN panel of ERP system with session, CRUD operations for employees table (generate empid and password) with salary calculation with mongoose, express and EJS template engine. Send email to employee when inserted. Encrypt the password. Logout.

ERP Admin Panel - Employee Salary Management

[+ Add New Employee](#) | [Logout](#)

Emp ID	Name	Email	Department	Basic	HRA (20%)	DA (10%)	PF (12%)	Net Salary	Actions
EMP001	Naitik Vaghasiya	naitik@erp.org	IT	60,000	12,000	6,000	7,200	70,800	Edit Delete
EMP002	Rahul Sharma	rahul@erp.org	HR	45,000	9,000	4,500	5,400	53,100	Edit Delete

Figure 4.1: ERP Admin Dashboard with Computed Salary (HRA 20%, DA 10%, PF 12%, Net Pay)

Add New Employee

EmpID and random password will be auto-generated, encrypted with bcrypt, and emailed.

Employee Name:

Email Address:

Department:

Basic Salary:

[Back to Dashboard](#)

Figure 4.2: Add Employee Form with Auto-Generated EmpID, Encrypted Password & Email Dispatch

Question 5: Employee Site Login with JWT, Profile & Leave Application

Technology Stack: React.js, Express.js, JWT, Mongoose

Requirement: Develop employee site (refer Q4) login with JWT, for employees with mongoose, express and React.js. Create a home page with following links: - Page 1) display employee profile. - Page 2) Application for leave (fields date, reason, grant yes/no) Add, List - Logout.

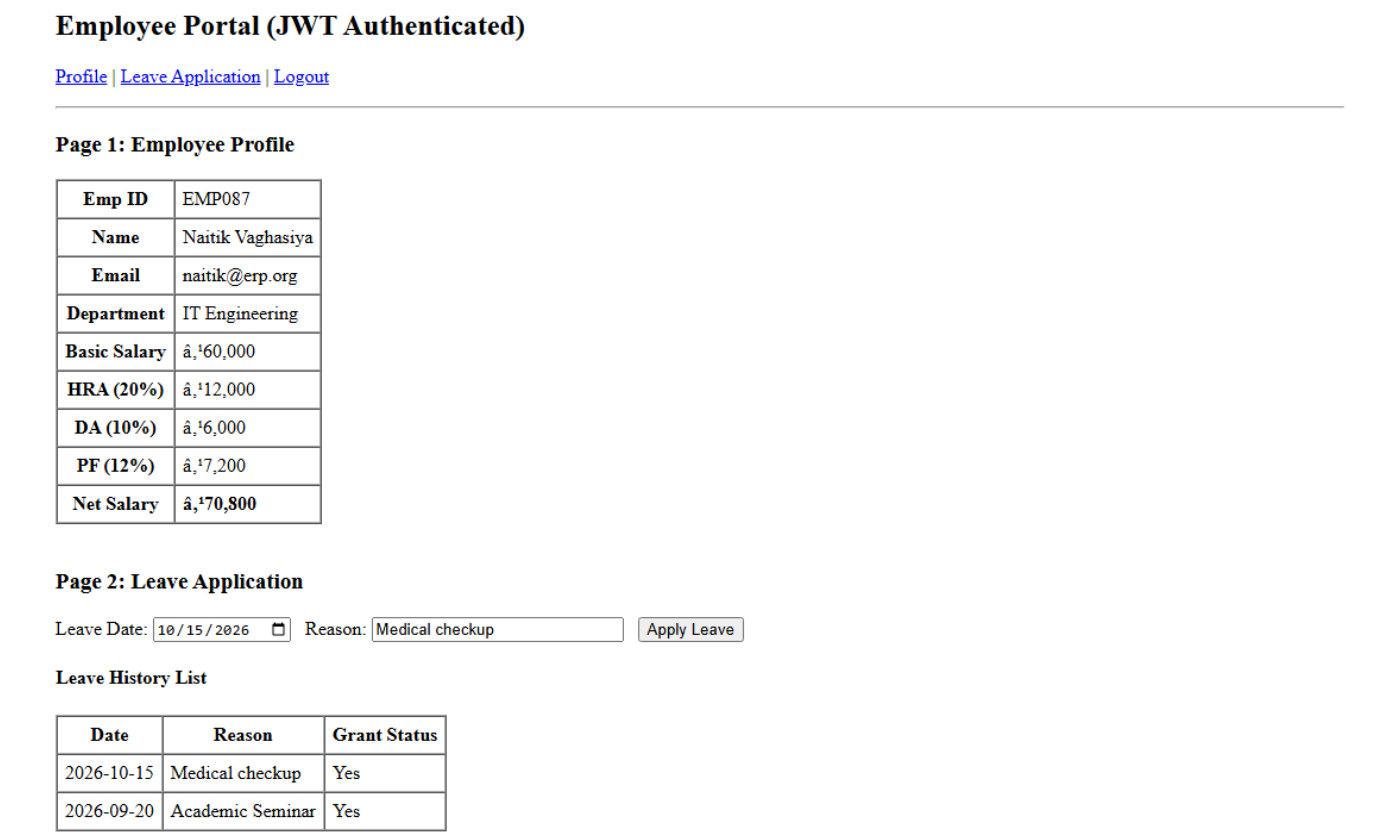


Figure 5.1: Employee Site - Page 1 Profile Display & Page 2 Leave Application (Add & List)

Question 6: Free API Utility Integration from Frontend and Backend

Technology Stack: [React.js](#), [Express.js](#), [Free Foreign Exchange API](#)

Requirement: Call any free api for a useful utility from frontend and backend.

Currency Converter (Free API Utility)

Calls Frankfurter Foreign Exchange API from Frontend via Express Backend

Amount:

From: To:

Result: 100 USD = 8374.50 INR

Figure 6.1: Currency Converter Utility calling Free API from Frontend via Backend

Question 7: MERN Shopping Cart with 2-Level Category & Products

Technology Stack: MERN Stack (MongoDB, Express, React, Node.js)

Requirement: Shopping cart with admin site and user site with (2 level) Category and Products using MERN stack.

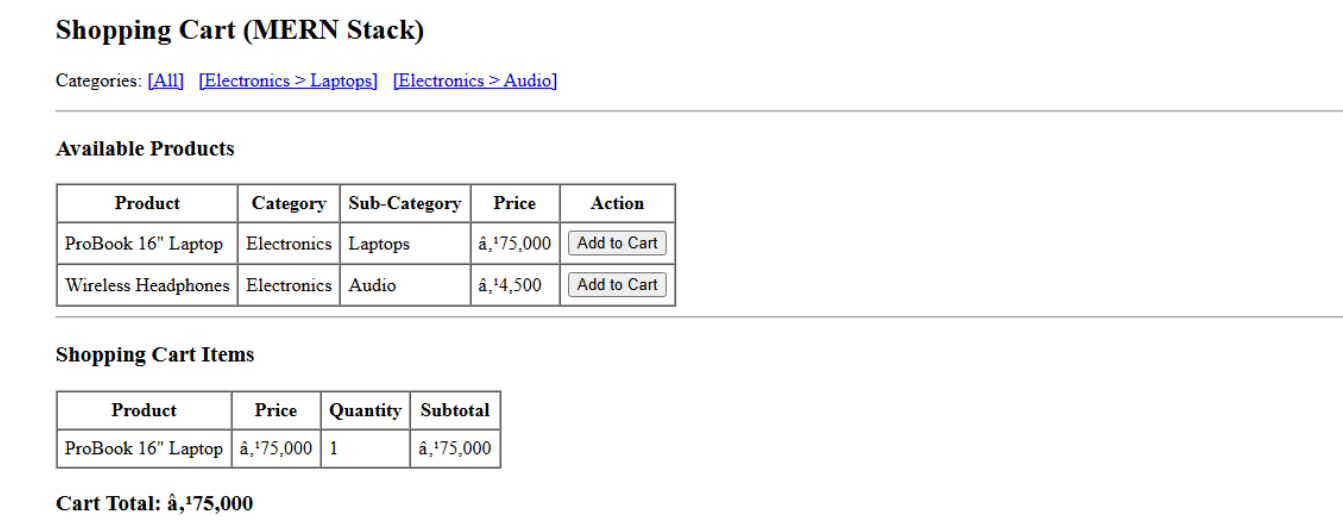


Figure 7.1: MERN Shopping Cart - Products by Category, Add to Cart & Live Total Calculation

Question 8: Student Collection CRUD using Sequelize ORM & React

Technology Stack: React.js, Express.js, Sequelize ORM

Requirement: Crud operation for Student collection using Sequelize. (Express + Sequelize + React)

Student Information System (Sequelize ORM CRUD)

Add Student

Name:

Email:

Age:

Course:

Student Directory

ID	Name	Email	Age	Course	Actions
87	Naitik Vaghasiya	naitik@university.edu	21	B.Sc. IT (Sem 7)	Edit Delete
88	Aarav Patel	aarav@university.edu	22	M.Sc. IT	Edit Delete

Figure 8.1: Student CRUD Operations with Sequelize ORM (Add Student Form & Student Directory Table)